

GEBZE TECHNICAL UNIVERSITY

Department of Computer Science

CSE 341 — Concepts of Programming Languages

D4 — AI Usage Journal

Part 2 Submission

22 May 2026

Halit Batuhan ASLAN

220104004003

Journal Contents

Part1;

[Entry #1 — Domain Selection \(Design\)](#) 29 April 2026

[Entry #2 — Language Naming \(Design\)](#) 1 May 2026

[Entry #3 — Lexer Implementation \(Lexer\)](#) 4 May 2026

[Entry #4 — Parser Implementation \(Parser\)](#) 4 May 2026

[Entry #5 — Test Suite & D3 Programs \(Testing\)](#) 4 May 2026

[Experiment \(E1\) — EBNF Grammar Generation \(Design/Parser\)](#) 1 May 2026

Part 2;

[Entry #1 — Consistency: §4.4 Semantics Audit](#) 16 May 2026

[Entry #2 — Consistency: §4.5, §4.7, §4.8 Audit](#) 16 May 2026

[Entry #3 — Interpreter Implementation](#) 16 May 2026

[Entry #4 — Stress Test Suite](#) 17-19 May 2026

[Experiment \(E2\) — Name equivalence and structural equivalence](#) ...16 May 2026

[Experiment \(E3\) — Implementing Ayar's type checker](#)..... 16 May 2026

Entry # 1 of 5 · Date 2026-04-29 · Phase Design · AI tool Claude Sonnet 4.6

Chat Links: <https://claude.ai/share/109c8e7f-4e81-438c-9713-a1070507b8ef>

Goal. Select an application domain for my DSL that fits all project requirements and aligns with my interest in machine learning.

Prompt. "I want to design a language focusing on ML technologies. The goal is to perform configuration and training simulations of machine learning models — users can define models (KNN, Decision Tree, etc.), feed them into a training block, obtain results, compare metrics (analyzing overfitting/underfitting), and select the best model. The language will not run heavy ML libraries; it will simulate training and metric generation. Explore this domain and check if it matches all project requirements."

Response (key part). AI confirmed every project requirement maps naturally: 3+ primitive types (int, float, bool, metric), structured type (experiment record), control structures (if, experiment block), user-defined functions, and a genuine domain-specific construct (compare...select best by). It highlighted that the "simulation" angle gives strong Sebasta §3.5 operational semantics material, and the metric type creates a well-motivated name equivalence discussion for §6.14.

Accepted. The ML model selection & training simulation domain — it covers all minimum requirements without forcing any feature, and the "compare by ... select best" construct is a clear DSL-identity argument that no generic language would have.

Rejected / modified. Earlier broader ideas (a general ML scripting language) were eliminated because the rubric penalizes languages indistinguishable from generic scripting. I also rejected narrower options (pipeline-only, data-only) because they would make the expression language too thin for Sebasta Ch. 7 discussions.

Errors caught. None in this session — the AI's domain analysis was accurate. I verified the requirement mapping myself against the handout checklist.

Reflection. AI is very useful for brainstorming and validating domain scope against a rubric, but the final domain choice must come from my own interest and judgment. The AI helped me see why "too broad = generic language" and "too narrow = thin expressions" are both traps.

Entry # 2 of 5 · Date 2026-05-01 · Phase Design · AI tool Claude Sonnet 4.6

Chat Links: <https://claude.ai/share/109c8e7f-4e81-438c-9713-a1070507b8ef>

Goal. Choose a name for my DSL that reflects the ML model selection domain and incorporates an old Turkish word.

Prompt. "Give me some name ideas which correspond to ML model selection correctly and also an old Turkish word."

Response (key part). AI proposed several candidates: Ayar (Ottoman Turkish for "calibration/tuning"), Seçer ("selector"), Örnek ("sample/model"), Elgen ("intelligent"), Sezip ("to sense/detect"), Sınav ("trial/test"). Each was evaluated for domain fit, pronounceability, and file-extension viability.

Accepted. "Ayar" — it means calibration/tuning in Ottoman Turkish, which maps directly to hyperparameter tuning. Short, clean, works globally as a file extension (.ayar), and has a one-sentence defensible explanation.

Rejected / modified. Eliminated Seçer, Sınav, Örnek because they contain Turkish-specific characters (ç, ı, ö) making them inaccessible internationally. Eliminated Sezip (good sound but weak domain fit) and Elgen ("gen" part doesn't match the language's purpose precisely).

Errors caught. No factual errors. However, the AI did not proactively flag the Turkish character problem — I caught it myself and applied the elimination. This is a usability concern the AI should have anticipated given I said "address to world."

Reflection. AI generates creative options quickly but doesn't always apply practical constraints unless told. I should state all constraints (no special characters, international audience) upfront in naming prompts rather than filtering afterward.

Entry # 3 of 5 · Date 2026-05-04 · Phase Lexer · AI tool Claude Sonnet 4.6

Chat Links: <https://claude.ai/share/fa555c75-262b-4816-8429-1c44e20fa024>

Goal. Generate a complete lexical analyzer (Lexer) for Ayar in Python, consistent with the finalized EBNF grammar and D1 design specification.

Prompt. "I am a university student working on a CSE341 course project. My language is called Ayar — a DSL for ML model configuration, training simulation, and model selection. [Full EBNF grammar pasted]. Write a clean, object-oriented Lexer class in Python. No external libraries (PLY, Lark, ANTLR). Strict requirements: Token dataclass with type/value/line/column; explicit keyword vs identifier separation; INT vs FLOAT distinction; comment and whitespace handling with correct line/column tracking; custom LexicalError with exact location; all separators and operators including two-char ones (==, !=, <=, >=). Include a test block with my sample program."

Response (key part). AI produced a well-structured ~180-line lexer with six sections: Token dataclass, LexicalError exception, keyword/metric/bool frozensets, a compiled master regex (TOKEN_SPEC), the tokenize() loop, and a sample program test. It used a single compiled regex with named groups and maximal-munch ordering. The WORD pattern matches all identifier-shaped strings, then a post-classification step checks frozensets to assign KEYWORD, METRIC_LITERAL, BOOL_LITERAL, or IDENTIFIER types. It produced 236 tokens from the sample program with no errors.

Accepted. The overall architecture — single compiled master regex with ordered alternatives implementing maximal munch (Sebesta §4.2), post-scan keyword classification via frozensets (Sebesta §3.3), Token dataclass with line/column tracking, and custom LexicalError. The pattern ordering logic was correct: COMMENT before OP_DIV, FLOAT before INT, two-char operators before single-char ones. The 10-test verification suite was also useful for confirming each token category.

Rejected / modified. (1) Identifier regex was `[A-Za-z_][A-Za-z0-9_]*` — allowing underscore as a first character. My EBNF says `<identifier> ::= <letter> { <letter> | <digit> | "_" }`, so underscore can only appear inside, not at the start. Fixed to `[A-Za-z][A-Za-z0-9_]*`. (2) The sample program included a NaiveBayes model with `var_smoothing = 1e-9` — scientific notation does not exist in my grammar's `float_literal` rule. Removed that model entirely. (3) The analyze block in the sample used `threshold = 0.05;` (semicolon followed by comma) but my EBNF's `analyze_field_list` has no comma separator — fields are simply juxtaposed, each self-terminated with `;`. Fixed the sample accordingly.

Errors caught. (1) Underscore-start bug — `_myvar` would have been silently accepted as a valid identifier, violating my lexical spec. Caught by tracing the EBNF identifier rule against the regex. (2) Scientific notation `1e-9` is not in the grammar but was used in the test program; the lexer would split it into four tokens (`INT "1"`, `IDENTIFIER "e"`, `OP_MINUS "-"`, `INT "9"`) which would then confuse the parser. Caught by cross-referencing the `float_literal` EBNF rule. (3) Comma-in-analyze was a sample-level error, not a lexer bug — the lexer would tokenize it fine, but the parser would reject it. Caught by comparing the sample against the finalized EBNF.

Reflection. AI produces solid lexer architecture quickly — the master-regex approach with maximal munch and post-classification is textbook-correct. However, it does not rigorously cross-check the regex patterns against the EBNF rules character by character; the underscore-start and scientific notation issues both came from the AI applying "common Python conventions" rather than reading my specific grammar. Lesson: always diff every regex in `TOKEN_SPEC` against the corresponding EBNF lexical rule before accepting.

Entry # 4 of 5 · Date 2026-05-04 · Phase Parser · AI tool Claude Sonnet 4.6

Chat Links: <https://claude.ai/share/77689341-3415-449d-8bbc-44307aa38130>

Goal. Generate a recursive-descent parser for Ayar that builds an AST, consistent with the finalized EBNF grammar and my existing lexer.

Prompt. "I am designing a DSL called Ayar for ML model configuration, training simulation, and model selection. I previously wrote the Lexer. Now write a clean, object-oriented Recursive-Descent Parser in Python. No external libraries. [Full EBNF grammar pasted]. Strict requirements: explicit AST node classes (`ProgramNode`, `ModelDeclNode`, `BinOpNode`, etc.); one parsing method per EBNF non-terminal; left-associative AST construction for arithmetic/logical operators; custom `ParseError` with expected token, actual token, and exact line/column; AST dump method for readable indented output. Assume tokens have `.type`, `.value`, `.line_number`, `.column_number` attributes."

Response (key part). AI produced a ~500-line parser with 20+ AST node classes, one `_parse_*` method per non-terminal, iterative while-loops for left-associativity (Sebesta §4.4), `LL(1)` dispatch via keyword lookahead, and a `_consume()` gate method that raises `ParseError` with full location info. It included a self-test block and passed on the sample program.

Accepted. The core recursive-descent architecture — one method per non-terminal with `LL(1)` dispatch — is textbook-correct (Sebesta §4.4.1). The left-associativity mechanism using while-loops (accumulating the left subtree as each operator is consumed) correctly avoids the infinite recursion that direct left-recursion would cause in a top-down parser. The `ParseError` design with exact location metadata via `_consume()` was also clean. Additionally, I accepted the layout validation pass (`validate_layout`) that enforces my curly-brace-on-its-own-line rule as a pre-parse step — this was a good separation of concerns since line-position constraints cannot be expressed in EBNF.

Rejected / modified. (1) Token type names mismatched my lexer in 8 places: AI used `ASSIGN` (my lexer emits `OP_ASSIGN`), `INT_LIT/FLOAT_LIT/STRING_LIT` (my lexer emits `INT_LITERAL/FLOAT_LITERAL/STRING_LITERAL`), and treated booleans/metrics as keywords rather than dedicated `BOOL_LITERAL` and `METRIC_LITERAL` types. All 8 were corrected by cross-referencing the parser's `_consume()` calls against my lexer's actual output. (2) `_parse_split_part` accepted `INT_LITERAL` for ratios — my EBNF says `<float_literal>` only, so `train(1)` should be a

ParseError. Fixed to reject integers. (3) `_parse_analyze_stmt` used commas between fields — my EBNF's `analyze_field_list` has no comma separator, fields are juxtaposed and each self-terminated with `;`. Rewrote the loop to continue while the next token is a recognized field keyword. (4) `_parse_eval_target` only handled plain identifier and bracket form — missing the dotted form comparison.best shown in D1's sample program. Added it as a third alternative.

Errors caught. (1) The 8 token-type mismatches caused immediate test failures — every `_consume("ASSIGN")` call crashed because the lexer emits `OP_ASSIGN`. Caught by running the parser with my actual lexer. (2) The split ratio bug would silently accept `train(1)` as valid, violating the grammar. Caught by tracing the EBNF rule. (3) The comma-in-analyze bug would reject my D1 sample program's analyze block. Caught by parsing the sample program end-to-end.

Reflection. AI produces excellent parser architecture (recursive-descent structure, precedence-via-call-depth, left-associativity via iteration) but is unreliable on integration details — it assumed token type names rather than reading my actual lexer output. The most important lesson: always run the AI-generated parser against your own lexer's real token stream before accepting anything, and cross-check every `_consume()` call against the lexer's token type catalog.

Entry # 5 of 5 · Date 2026-05-04 · Phase Testing · AI tool Claude Sonnet 4.6

Chat Links: <https://claude.ai/share/c3142c7b-733c-4111-8055-8aa7458a9670>

Goal. Generate valid and invalid Ayar test programs (D3 deliverable) and a test runner script to verify the lexer and parser work correctly end-to-end.

Prompt. "I have attached my Lexer, Parser, and D1 Design Spec with EBNF grammar. My next deliverable is D3 — Example Programs & Test Report. Generate 3 valid .ayar programs that together exercise every language feature (dataset, splits, all model types, all experiment statements, report). Generate 5 invalid .ayar programs each with a deliberate syntax error. Write a `run_tests.py` script that imports my Lexer and Parser, tests valid programs by printing the AST, and tests invalid programs by catching `LexicalError/ParseError` with line and column info."

Response (key part). AI produced 3 valid programs (`sample_1` covering KNN+DecisionTree with bracket eval, `sample_2` covering SVM+NaiveBayes+LogisticRegression with `lower_is_better`, `sample_3` covering all five model types), 5 invalid programs (missing semicolon, unknown model type, invalid split role, inline brace layout violation, unknown keyword), and a 34-test runner script. All passed initially.

Accepted. The test runner architecture — a simple custom script with no external frameworks, using `try/except` to catch `ParseError` and `LayoutError` with fragment-matching assertions. The 3 valid programs together cover every grammar rule: all 5 model types, all split roles, all experiment statements (`train`, `evaluate`, `collect`, `analyze`, `compare`, `exclude`, `select`), both compare directions, all three `eval_target` forms (plain, dotted, bracket), and both report field types. The isolation tests (V1–V12) testing one feature each were also useful for pinpointing failures.

Rejected / modified. (1) The initial test files used commas between analyze fields (`threshold = 0.10;`) because the AI based them on an older parser version. After I updated the parser to drop commas from `analyze_field_list` (fields now juxtaposed, each self-terminated with `;`), all analyze blocks in the 3 valid programs and all inline tests had to be regenerated. (2) The initial tests did not cover the dotted `eval_target` form (`evaluate comparison.best on iris.test;`) because the first

parser version lacked it. After I added that form to the parser, a new valid test (V9) was added and sample_1 was updated to use it. (3) I had to re-request the full test suite twice because the AI kept generating tests against a stale version of the parser rather than re-reading the uploaded files from scratch.

Errors caught. (1) The AI's first test suite passed 33/33 against an old parser but failed against my updated one — specifically every analyze block test broke because the old `;` separator was now a `ParseError`. Caught by running the tests after updating the parser. (2) An invalid test case (I11 — empty analyze body) was missing initially; the updated parser explicitly raises an error for analyze overfitting `{ }` with no fields, so a dedicated failure test was needed. Caught by manually testing edge cases.

Reflection. AI generates comprehensive test suites quickly, but they become stale the moment you change the parser or grammar. The key lesson: after any EBNF or parser modification, re-run the entire test suite immediately and re-generate affected tests. Also, providing the AI with both the grammar and the actual parser source (not just the grammar) is essential — otherwise it generates tests against what the grammar says, not what the code actually does.

Experiment 1 -> E1 · Date 2026-05-01 · Phase Parser / Design · AI tool Claude Sonnet 4.6

Chat Links: <https://claude.ai/share/9ef8e86e-e704-49ac-8260-a7a55046f212>

Goal: Generate complete EBNF grammar for Ayar following Sebesta §3.1–3.2 style (Experiment E1).

Prompt. "I am a university student working on a CSE341 course project. My language is called Ayar — a DSL for ML model configuration, training simulation, and model selection. [Full example program pasted]. Design decisions already fixed: primitive types `int`, `float`, `bool`, `metric`; structured type `experiment`; static scoping; name equivalence for `metric`; `int`→`float` coercion only; assignment is a statement; short-circuit for `or`/`and`; intended precedence low→high: `or` → `and` → `not` → `==` `!=` `<` `<=` `>` `>=` → `+` `-` → `*` `/` → unary minus → primary. YOUR TASK — Generate a complete unambiguous EBNF grammar covering: program structure, dataset/split, model declaration, experiment block (`train`, `evaluate`, `collect`, `analyze`, `compare`, `exclude`, `select`), report, expression language with correct precedence, all types/literals. After generating, provide: operator precedence table, ambiguities found, assumptions list, and two design decisions to reconsider."

Response (key part). AI produced a full EBNF grammar with 40+ productions using stratified nonterminals for expression precedence (eight levels deep). It identified 5 ambiguities (most notably the `or` keyword collision between `exclude_condition` and boolean expressions) and listed 10 assumptions (A1–A10). It recommended two design changes: (D1) make model types open identifiers instead of grammar-level keywords, (D2) drop the bracket syntax `[comparison.best]` in favor of plain dot notation.

Accepted. The overall stratified expression grammar structure encoding precedence through nonterminal depth — this correctly implements Sebesta §3.3's technique. The `or` keyword collision resolution (giving `exclude_condition` its own closed sub-grammar separate from `<expr>`) was a genuine ambiguity I had missed. The recommendation D2 to simplify `eval_target` notation was also adopted.

Rejected / modified. (1) The AI used commas as separators inside the analyze block, but my parser actually uses semicolons as terminators on each field with no separator between fields. I corrected `analyze_field_list` to `::= <analyze_field> { <analyze_field> }` and `analyze_field` to end with `;"`. (2) The AI's initial grammar omitted the plain dotted form `<identifier> "." <identifier>` from `<eval_target>` — only had plain identifier and bracketed form. I added the dotted form as a middle alternative so the parser tries the longer match first. (3) I kept model types as closed keywords (rejected D1) because my parser validates them at syntax level and I can defend this choice as a deliberate DSL constraint in the exam.

Errors caught. (1) Separator inconsistency in analyze block — AI applied the comma convention from model blocks without checking that analyze uses a different pattern (semicolon-terminated, no separator). Caught by comparing the generated EBNF against my actual parser code. (2) Missing `eval_target` alternative — the dotted form `comparison.best` was absent, meaning a valid program would be rejected. Caught by tracing a test program through the grammar by hand.

Reflection. AI generates structurally sound grammars quickly, especially the precedence-encoding layered nonterminals, but it unifies separator conventions across all blocks even when the actual design differs block by block. The lesson: always trace at least one complete example program through the generated grammar rule-by-rule before accepting it. Also, providing my own precedence table in the prompt made error detection much easier — without it I would not have had a clear baseline to compare against.

Part2;

Entry #1 of 4 · Date: 2026-05-16 · Phase: Design / Writing · AI tool: Claude Sonnet 4.6

Chat Links: <https://claude.ai/share/d4195088-717f-4165-9fda-353cad0163c2>

Goal: Verify whether §4.4 (Semantics) is internally consistent and consistent with §4.1, §4.3, §4.6, and the D2 parser/AST structure.

Prompt: "I have attached my updated D1 Design Specification and my D2 implementation source code. I want you to check whether §4.4 (Semantics) is consistent with all other sections of D1. Specifically: (1) Does the state triple (Γ, M, R) align with the scope and lifetime descriptions in §4.6? (2) Do the semantic rules cover every construct in the EBNF grammar? (3) Does the worked example trace follow correctly from the rules? (4) Does §4.4 contradict anything in §4.1? (5) Do the semantic rules contradict anything visible in the D2 parser or AST structure?"

Response (key part): AI identified one genuine inconsistency in Q1: the E-ExperimentBlock conclusion outputs $(\Gamma[\text{exp_name.best} \mapsto R'], M', \perp)$ — M' survives in the output state — while both the prose explanation and §4.6 explicitly state that M is stack-dynamic and discarded when the block closes. AI correctly identified four constructs with no semantic rules (`dataset_decl`, `split_stmt`, `eval_stmt`, `report_stmt`) and flagged that the absence of a `dataset_decl` rule leaves E-Train's precondition $\text{dataset_ref} \in \text{dom}(\Gamma)$ formally ungrounded. For Q3, AI claimed no worked trace existed in §4.4.

Accepted:

- Q1 inconsistency — E-ExperimentBlock formula bug is genuine. Fixed conclusion from M' to M (restoring outer M , discarding inner M').
- Q2 `dataset_decl` dependency — genuine incompleteness. Added two minimal supporting rules (E-DatasetDecl and E-SplitStmt) to ground E-Train's precondition.
- Q4 and Q5 — consistent, no changes needed.

Rejected / modified:

- Q3 claim that no worked trace exists — incorrect. The worked example trace was present in §4.4 showing σ_0 through σ_{final} for the `iris_exp` program. AI missed it during its document read. Pushed back explicitly in follow-up message.
- Q2 recommendation to add rules for `report_stmt` and top-level `eval_stmt` — rejected. The handout explicitly allows formalizing only two constructs. Only the `dataset_decl` rule was added as the minimum necessary to ground existing preconditions.

Errors caught:

- E-ExperimentBlock formula bug — caught by AI correctly. The formal rule and the prose directly contradicted each other. Fix: $\rightarrow (\Gamma[\text{exp_name.best} \mapsto R'], M, \perp)$.
- AI missed the worked trace section — caught by cross-checking AI's claim against the actual document. Demonstrates that AI document reading can be incomplete even on short documents.

Reflection: AI is effective at cross-referencing formal rules against prose descriptions and identifying contradictions between them. The E-ExperimentBlock bug — where the formula contradicted its own explanation — is exactly the kind of inconsistency a human reviewer would easily overlook. However, AI document reading is not exhaustive: it missed an entire section of §4.4. The lesson: use AI audit findings as a starting point, not a final verdict. Always verify "this does not exist" claims by checking the document manually.

Entry #2 of 4 · Date: 2026-05-16 · Phase: Design / Writing · AI tool: Claude Sonnet 4.6

Chat Links: <https://claude.ai/share/d4195088-717f-4165-9fda-353cad0163c2>

Goal: Verify whether §4.5 (Type System), §4.7 (Expressions and Assignment), and §4.8 (Design Rationale) are internally consistent and backed by the rest of D1.

Prompt: "Now please check the remaining Part 2 sections: §4.5, §4.7, and §4.8. (1) Does metric $\neq$ float name equivalence align with the lexer and grammar? (2) Does the int $\rightarrow$ float coercion rule appear in §4.4? (3) Does the precedence table match the EBNF exactly? (4) Does anything implicitly assume short-circuit behavior? (5) Does each §4.8 design decision have a corresponding technical claim in §4.4, §4.5, or §4.7?"

Response (key part): AI confirmed the lexer is fully consistent with §4.5 — metric literals are correctly classified as a distinct token type. It identified that `<primary>` permits `<metric_literal>` in arithmetic positions without grammatical restriction, meaning the metric $\neq$ float boundary has no syntactic enforcement and relies entirely on the type checker. For Q3, it found one genuine terminology error: not and unary minus are called "right-associative" in §4.7, but Sebesta §7.3 defines associativity only for binary operators — the correct term is "right-recursive." For Q5, it confirmed that Decisions 4 (assignment-as-statement) and 3 (encapsulation, post-fix) are well-backed, while Decision 2's coercion sub-claim has no formal grounding in §4.4.

Accepted:

- Q1 lexer consistency — confirmed fully consistent. No change needed.
- Q3 terminology error — not and unary minus corrected from "right-associative" to "right-recursive" in §4.7 precedence table. Sebesta §7.3 is explicit that associativity is a binary operator property.
- Q4 — confirmed no implicit short-circuit assumption. P-Exclude operates on pre-computed tags. The `<exclude_condition>` sub-grammar is structurally incapable of short-circuiting.

Rejected / modified:

- Q2 "coercion absent from §4.4 is a problem" — rejected. §4.4 formalizes only two constructs; the expression sub-language is intentionally out of scope. Coercion is stated as design intent in §4.5; formal grounding is in the D2 implementation.
- Q5 enforcement gap for Decisions 1 and 2 — partially rejected. The absence of $\Gamma \vdash e : T$ judgement rules is a scope decision consistent with the handout requirements, not a documentation failure. The D2 type checker provides the formal enforcement.
- Q4 rhetorical imprecision claim — partially rejected. §4.7 uses exclude if overfit or underfit as a motivating example for full evaluation, then extends the argument to the

general expression language. AI called this "conflating two distinct arguments." This is accurate as a criticism but the conclusion (full evaluation is correct) is unaffected.

Errors caught:

- not terminology error — caught by AI correctly. Sebesta §7.3 is unambiguous: associativity applies only to binary operators. "Right-recursive" is the precise term for unary operator chaining direction.
- AI's Q3 note that = is absent from the precedence table — valid observation but not a bug. §4.7 explicitly states assignment is a statement, so its absence from the expression operator table is correct by design. No change made.

Reflection: AI cross-referencing a precedence table against stratified EBNF nonterminals is highly reliable — it correctly matched all seven levels without error. Terminology precision is also a strength: catching "right-associative" vs "right-recursive" required knowing Sebesta §7.3's exact definition of associativity. The weakness in this session was distinguishing between "formally unanchored" and "out of scope by design" — AI flagged the absence of type judgement rules as a gap, but the handout explicitly allows this. Understanding what the grader actually requires versus what formal completeness would require is a judgment call that AI cannot make on its own.

Entry #3 of 4 • Date: 2026-05-16 • Phase: Interpreter • AI tool: Claude Sonnet 4.6

Chat Links: <https://claude.ai/share/fa776364-38f9-4e96-87bd-acb9fd2d4935>

Goal: Implement a complete tree-walking interpreter for Ayar that faithfully follows the state triple (Γ, M, R) from D1 §4.4, using Option C formula-based simulation for deterministic metric generation.

Prompt: "I now need to implement an interpreter for Ayar in Python. Before writing any code, I want to discuss one key design decision — the simulate() function. Option A uses hash-based determinism; Option C uses model-type formulas where hyperparameter values directly affect output metrics. Which approach do you recommend? [After confirmation of Option C:] Please write the full interpreter. Architecture: state triple (Γ, M, R) from D1 §4.4, one formula per model type, train adds 0.07 accuracy bonus, evaluate uses base formula, all statement types implemented, runtime errors DataLeakageError / NoEligibleModelError / UndefinedModelError, TypeChecker called first, entry point Interpreter(ast, tokens).run()."

Response (key part): AI recommended Option C, arguing that interpretability and exam defensibility strongly favor formulas over hashing — "if asked why increasing max_depth improves accuracy, you can explain the ML concept behind it; with Option A your answer is essentially the hash changed." AI then produced a ~990-line interpreter implementing the full state triple, all statement execution methods, the simulate() function with five model-type formulas, and formatted report output. During testing, two bugs were found and fixed.

Accepted:

- Option C recommendation — fully agreed. Formula-based simulation makes hyperparameter effects directly visible and defensible in an exam setting. Hash-based simulation would be semantically opaque.

- State triple implementation — `self._gamma` (Γ), `self._M` (M), `self._R` (R) map exactly to the D1 §4.4 formal definitions. M is created empty at experiment start and discarded at block end, consistent with stack-dynamic lifetime in §4.6.
- `eval_accuracy` field fix — when `collect metrics` restricts the metrics dict, `analyze overfitting` was reading `metrics.get("accuracy", 0.0)` which returned 0.0 after accuracy was filtered out, incorrectly tagging all models as underfit. Adding a dedicated `eval_accuracy` field to `ModelResult` that is never touched by `collect` was the correct fix.
- Runtime error handling — `AyarRuntimeError` base class with `DataLeakageError`, `NoEligibleModelError`, `UndefinedModelError` as subclasses. Clean formatted output via `try-except` in `run_source()`.

Rejected / modified:

- Demo program threshold — initial `_DEMO_PROGRAM` used `threshold = 0.05` but the fixed `_TRAIN_BONUS` is always 0.07, so all models were tagged as overfit. Fixed threshold to 0.10. This is not an interpreter bug — it is a demo configuration error.
- `sys.argv` not connected — interpreter always ran `_DEMO_PROGRAM` regardless of command-line argument. Added proper `if __name__ == '__main__':` block using `sys.argv` to read a file from disk.
- `sample_full.ayar` naming discrepancy — experiment named `iris_comparison` but eval target said `comparison.best`. This was a stale name in the sample file, not an interpreter bug. Fixed by renaming eval target to `iris_comparison.best`.

Errors caught:

- `eval_accuracy` bug — caught by running `sample_2.ayar` which collects `[precision, recall, f1]` without accuracy. All models were incorrectly tagged as underfit because `analyze overfitting` read `metrics.get("accuracy", 0.0) = 0.0 < 0.70`. Traced by adding debug output to `_exec_analyze_stmt` and comparing the metrics dict against expected values.
- `sys.argv` not wired — caught by running `python ayar_interpreter.py sample_1.ayar` and observing the demo program output instead of `sample_1` output.

Reflection: AI interpreter architecture is structurally sound — the state triple mapping and visitor-style dispatch are clean and consistent with D1 §4.4. The most valuable part of this session was the `simulate()` design discussion: being forced to choose between Option A and C and defend the choice mirrors exactly the kind of reasoning the exam will require. The `eval_accuracy` bug illustrates a subtle interaction between two statements (`collect` and `analyze`) that only surfaces with specific programs — single-experiment demos would never expose it. Lesson: always test with programs that use every statement combination, not just the happy path.

Entry #4 of 4 • Date: 2026-05-17-19 • Phase: Testing • AI tool: Claude Sonnet 4.6

Chat Links: <https://claude.ai/share/ae290711-8f81-43db-821a-7888ab29535f>

Goal. Generate a comprehensive automated stress test suite covering all four Ayar components across two rounds of testing, find genuine bugs, apply fixes, and produce a final 28/28 passing test harness that auto-discovers test files by naming convention.

Prompt (Round 1). "I need you to generate a comprehensive stress test suite for Ayar. Generate 20 tests across 4 categories: Lexer edge cases (5), Parser stress tests (5), Type checker stress tests (5), Interpreter stress tests (5). For each test show the .ayar source, expected output, and PASS/FAIL. Run all tests against my actual implementation."

Prompt (Round 2). "Generate 20 new .ayar test programs different from the first round. [Categories listed.] Run all against the patched implementation."

Prompt (Final harness). "Rewrite the test suite from scratch to include ALL files in my directory. Auto-discover files by name prefix: 2l→lexer, 2p→parser, 2tc→typechecker, 2i→interpreter, sample_→end-to-end. Infer pass/fail from whether errors are raised, do not hardcode expected outputs. Keep --verbose, --stop-first, --category flags."

Response (key part). Round 1: 17/20 pass, 3 bugs found (KNN/DT formula collision, TypeChecker misleading docstring, Parser zero-field model). Round 2: 17/20 pass, 3 new bugs found (block comment lexer support missing, report always showing last experiment's M, model/dataset name collision not detected). After all fixes applied: final harness 28/28 pass on first run. The script auto-discovers all .ayar files by naming convention and classifies expected outcome from filename alone — no hardcoded outputs.

Accepted.

- Auto-discovery architecture — reading filename prefix to infer expected outcome is robust and extensible. New test files are picked up automatically without changing the script.
- Round 1 I2 fix — KNN base changed 0.95→0.94, eliminating the collision with DecisionTree at k=5/max_depth=5 where both produced accuracy=0.85.
- Round 2 L1 fix — BLOCK_COMMENT token added to TOKEN_SPEC before OP_DIV with `[\s\S]*?` pattern for multi-line support and correct `\n` counting for line tracking.
- Round 2 P1 fix — `_exec_experiment()` stores M snapshot as `exp_name.__metrics__` in `Γ`; `_exec_report_stmt()` uses three-tier lookup: direct name match → prefix substring scan → last experiment fallback.
- Round 2 TC3 fix — R6-NameCollision added using set intersection of `_declared_models` and `_declared_datasets`.

Rejected / modified.

- Round 1 TC2 "model declared after experiment is a bug" — rejected. Two-pass forward scan is intentional hoisting semantics documented in §4.6. Only the misleading "declare-before-use" docstring was corrected.
- Round 1 P2 "fix the parser for zero-field models" — rejected. EBNF `<field_list> ::= <field> { ";" <field> }` requires minimum 1 field by design. Added comment only.
- Initial test harness used hardcoded expected outputs — rejected in favor of filename-based classification. Hardcoded outputs would break whenever implementation details changed.

Errors caught.

- KNN/DT formula collision — caught by T17 requiring all five model types to produce distinct accuracy values. Python dict insertion order would determine winner non-deterministically.
- `_last_experiment_M` overwrite — caught by T20 running two experiments and verifying each named report shows its own experiment's winner. Single-experiment programs never expose this.
- Block comment tokenized as `OP_DIV OP_MUL ... OP_MUL OP_DIV` — caught by T03. Parser failure message was about unexpected identifiers rather than a clear "block comment not supported" error, making debugging difficult.
- `BLOCK_COMMENT` \n counting — initial fix used `col += len(lexeme)` ignoring embedded newlines. Caught by sub-test D verifying line numbers after a 4-line block comment.

Reflection. Two rounds of stress testing found 6 bugs total. The most valuable pattern: bugs that only appear in multi-component interaction (collect→analyze accuracy dependency, two-experiment report M overwrite) were only found because the test programs were specifically designed to exercise combinations rather than individual features. Single-path testing would have missed both. The auto-discovery harness is a genuine improvement over hardcoded tests — it scales as the test suite grows and forces the classification logic to be explicit in the filename rather than buried in test code. Lesson: design test programs around statement combinations, not individual features.

Experiment 1 -> E2 · Date: 2026-05-16 · Phase: Type Checker / Design · AI tool: Claude Sonnet 4.6

Chat Links: <https://claude.ai/share/65957183-eb11-496a-b778-406b415e2b0e>

Goal: Verify whether Ayar's name equivalence decision for the metric type is correctly grounded in Sebesta §6.14, and whether the enforcement claim in §4.5 logically follows from the equivalence rule.

Prompt: "I am a university student working on a DSL called Ayar for CSE 341 — Concepts of Programming Languages. I have attached my D1 Design Specification. My language has a type called metric whose value set is $\mathbb{R}$, identical to float. In §4.5 I declared that Ayar uses name equivalence for type checking, meaning metric and float are distinct types even though they have the same underlying representation. I want you to do the following: (1) Explain the difference between name equivalence and structural equivalence as defined in Sebesta §6.14. Use Ayar's metric vs float situation as your concrete example. (2) Under structural equivalence, would `accuracy + 0.5` be a type error in Ayar? Under name equivalence? Explain why the answers differ. (3) Give a concrete example of a silent bug that structural equivalence would allow in Ayar but name equivalence would catch. Use actual Ayar syntax in your example. (4) My D1 §4.5 says: 'the type checker rejects any operation that crosses this boundary.' Does this claim follow correctly from name equivalence, or is there a gap between the equivalence rule and the enforcement claim?"

Response (key part): AI correctly explained that structural equivalence looks inside the type definition (both metric and float are $\mathbb{R}$, so they would be compatible), while name equivalence stops at the name tag (metric $\neq$ float regardless of representation). For Q2, it correctly identified that `accuracy + 0.5` is not a type error under structural equivalence but is under name equivalence. For Q4, it identified a genuine gap: name equivalence alone only establishes that

the types are distinct — it does not automatically imply that mixing them in an expression is rejected. A separate operator compatibility rule is needed to close the gap. It correctly traced the three-step logical chain: (1) name equivalence $\rightarrow$ metric $\neq$ float; (2) no coercion rule for metric $\rightarrow$ float; (3) operator requires operand-type identity $\rightarrow$ rejection.

Accepted:

- Q1 and Q2 explanations — fully consistent with Sebesta §6.14, used correctly in the Ayar context.
- Q4 enforcement gap analysis — genuinely useful. The gap is real: §4.5 makes an enforcement claim that is a conclusion drawn from rules spread across two sections. Added one sentence to §4.5 to close the gap explicitly: *"Because no implicit coercion exists from metric to float (§4.7), and all operators require operand-type identity, any expression mixing the two types is rejected by the type checker."*

Rejected / modified:

- Q3 syntax example — the line threshold = accuracy in the analyze overfitting block would be a **parse error** in Ayar, not a type error. The `<analyze_field>` grammar rule only accepts `<float_literal>` on the right-hand side — accuracy (a METRIC_LITERAL token) would be rejected by the parser before the type checker ever runs. AI generated this example without cross-checking the grammar, applying general reasoning rather than reading the actual EBNF. The example is motivationally correct but syntactically invalid in Ayar.

Errors caught:

- Grammar mismatch in Q3 example — caught by tracing threshold = accuracy through `<analyze_field> ::= "threshold" "=" <float_literal> ";"`. Since accuracy is a METRIC_LITERAL and not a `<float_literal>`, the parser rejects it immediately. This also confirms why R3 (metric-in-arithmetic) can only be triggered via synthetic AST in the type checker test suite — the grammar prevents metric literals from appearing in any position where arithmetic would occur.

Reflection: AI explains Sebesta type system concepts at textbook level and correctly identifies real gaps between design claims and their formal grounding. However, it generates code examples by applying general reasoning rather than tracing through the actual grammar. In this session, that produced a syntactically invalid Ayar example for Q3. The lesson: for any AI-generated Ayar code example, manually trace it through the EBNF before accepting it as a valid illustration of a language property.

Experiment 2 -> E3 · Date: 2026-05-16 · Phase: Type Checker · AI tool: Claude Sonnet 4.6

Chat Links: <https://claude.ai/share/585a8324-6920-4d39-800d-eea455c84850>

Goal: Implement a complete type checker for Ayar in Python that enforces the five type rules declared in §4.5 of D1, with accurate line number reporting and a clean separation from the parser and interpreter.

Prompt: "I am a university student working on a DSL called Ayar for CSE 341 — Concepts of Programming Languages. I have attached my updated D1 Design Specification and my D2 implementation (lexer + parser source code). I need you to implement a type checker for Ayar in Python. The type checker receives the AST produced by my parser and checks semantic

constraints before execution. Type rules to enforce: (1) Every model name referenced in train or evaluate must be declared at global level. (2) Every dataset reference must refer to a declared and split dataset. (3) metric literals must not appear in arithmetic positions. (4) The metric named in compare by must have been declared in the preceding collect metrics. (5) Training on the test split is a data leakage error. Error format: each type error must report the type of violation and the line number. Architecture: separate TypeChecker class, takes ProgramNode, returns list of TypeErrors, must not modify the AST. Do not use any external libraries."

Response (key part): AI produced a two-pass type checker: Pass 1 (`_collect_globals`) scans all top-level declarations to build sets of declared datasets, split datasets, and model names. Pass 2 (`_check_node`) performs a recursive visitor-style walk enforcing all five rules. Since AST nodes do not carry line numbers, AI designed a `_LineMap` class that recovers line numbers by scanning the token stream for recognizable patterns. All three initial test inputs passed on first run.

Accepted:

- Two-pass architecture — separating global declaration collection from rule enforcement is clean and maps directly to the declare-before-use semantics in §4.6.
- `_LineMap` token stream recovery — the correct engineering solution given that the parser does not embed line numbers in AST nodes.
- R3 synthetic AST test — since the parser rejects accuracy + 0.5 at syntax level, hand-crafting the AST to test R3 in isolation is the standard compiler engineering approach. Accepted and documented as a known limitation.
- Import block with per-rule annotations — makes the dependency between each node type and its rule explicit.

Rejected / modified:

- `TypeError` class name — shadows Python's built-in `TypeError`. If any internal `isinstance()` check raised a genuine Python `TypeError`, the traceback would be deeply confusing. Renamed to `AyarTypeError` throughout all nine call sites, return type annotations, and docstrings.
- Missing R1 test with real parsed input — the initial test suite covered R1 only through synthetic AST. Requested an additional test (`ghost_model` referenced but never declared) that exercises `_LineMap.find_keyword_then_identifier` end-to-end with a live token stream. Both usages of `ghost_model` were caught at the correct source lines (18 and 21).

Errors caught:

- `TypeError` name shadowing — caught by recognizing it as a Python built-in. Would not have caused a crash in normal operation but would have produced deeply confusing tracebacks during debugging.
- `find_dataset_ref` token type imprecision — the original version matched the split role by value only (`role_val == role`) without checking `role_type == "KEYWORD"`. Split role tokens (train, validation, test) are emitted as `KEYWORD` by the lexer, not `IDENTIFIER`. The match would have worked by accident in most cases but would silently fail if a user-defined identifier shared a name with a role keyword. Fixed by adding the explicit type guard.

- R3 unreachable on real parsed input — caught by tracing accuracy + 0.5 through `_parse_field_value`, which only accepts `int` | `float` | `string` | `bool` literals. This means R3 is defensive dead code under the current grammar. Documented as a known limitation: R3 would activate if the grammar were extended to allow expressions in field positions.

Reflection: AI produces solid type checker architecture quickly — the two-pass visitor structure is textbook-correct and maps cleanly to the semantics declared in §4.6. However it does not check for naming collisions with Python built-ins, and it generates test suites against the grammar specification rather than the actual parser behavior. The most important lesson from this session: always run the generated code against your own lexer and parser immediately, and verify that error class names do not shadow Python built-ins before accepting any implementation.